To create a new Socket.io server in the NodeJS applications:

```
const { Server } = require("socket.io");

const io = new Server(3000, { cors: "http://localhost:5173" });
```

==========

To init the connection with Socket.IO:

```
io.on("connection", (socket) => {
    socket.on("disconnect", () => {
        onlineUsers = onlineUsers.filter(user => user.socketId != socket.id);

        io.emit("getOnlineUsers", onlineUsers);
    });
})
```

==========

To add a new user to the array, Redis DB, etc., after the connection from the client, the client emits the addNewUser event:

```
socket.on("addNewUser", (userId) => {
 // TODO: We should handle the case when the user has multiple devices.
      onlineUsers.push({
            userId,
            socketId: socket.id,
      });
      io.emit("getOnlineUsers", onlineUsers);
});
```

==========

To send the data to all sockets from the Socket.io server:

```
io.emit("getOnlineUsers", onlineUsers);
```

==========

To listen to the socket events from the client. And to send the data to a specific user from the server:

```
    socket.on("addMessage", (message) => {
        const user = onlineUsers.find(t1 => t1.userId === message.recipientId);
        if (user) {
            io.to(user.socketId).emit("getMessage", message);
            console.log("------------------------------------------------------
");
            console.log("The sender ID is: ", message.sender?._id);
            console.log("------------------------------------------------------
");
```

```
            io.to(user.socketId).emit("getNotification", {
                senderId: message.sender?._id,
                isRead: false,
                date: new Date(),
            });
        } else {
            console.log("------------------------------------------------------------
");
            console.log("The User is Not Online");
            console.log("------------------------------------------------------------
");
        }
    });
```

==========

To handle the disconnect from the client socket:

```
// console.log("The Connected Sockets are: ", Object.keys(io.sockets.sockets));
socket.on("disconnect", () => {
        onlineUsers = onlineUsers.filter(user => user.socketId != socket.id);

        io.emit("getOnlineUsers", onlineUsers);
});
```

==========

To use Socket.IO with React applications, we can use:

```
import { Socket, io } from "socket.io-client";
```

==========

To use the socket-object in the React application:

```
const [socket, setSocket] = useState<Socket>();
```

==========

To use the socket in React state management:

```
useEffect(() => {
        let newSocket = io("http://localhost:3000");
        setSocket(newSocket);

        return () => {
            if (socket?.connected) {
                socket?.off("getOnlineUsers");
                socket?.off("getMessage");
                socket?.off("getNotification");
                localStorage.setItem("currentChat", "");
            }
        };
    }, [user]);
```

==========

To listen to the Socket.IO events:

```
        socket?.on("getMessage", (message: MessageCD) => {
            const t1 = localStorage.getItem("currentChat");
            const t2 = t1 ? JSON.parse(t1) : null;

            if (t2?._id === message?.chat?._id) {
                if (!messages) {
                    setMessages([message]);
                } else {
                    setMessages((prev: Message[]) => [...prev, message]);
                }
            }
        });
```

==========

To send an event from the client to the server, we can use emit-method:

```
    useEffect(() => {
        if (socket) {
            let recipient = currentChat?.members.find(t1 => t1._id !== user._id);
            if (!recipient) return console.log("No Recipient Found For
NewMessage: ", newMessage);
            socket.emit("addMessage", { ...newMessage, recipientId: recipient._id
});
        }
    }, [newMessage]);
```

==========